MixBytes()

[SECURITY AUDIT REPORT]

Catapult PULT OFT Bridge

PREPARED FOR CATAPULT

REPORT DATE

November 10, 2026

[CONTENTS]

Table of Contents

1. Introduction 3
    1.1 Disclaimer 3
    1.2 Executive Summary 3
    1.3 Project Overview 5
    1.4 Security Audit Methodology 7
    1.5 Risk Classification 9
    1.6 Summary of Findings 10
2. Findings Report 10
    2.1 Critical 10
    2.2 High 10
    2.3 Medium 10
    2.4 Low 11
        L-1 Recipient check misses non-canonical to encodings 11
3. About MixBytes 12

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Catapult Trade (PULT) is an ERC-20 token with a fixed supply of one billion tokens, and the audited bridge makes it available on other EVM chains through LayerZero V2. `PultOFTAdapter` locks PULT on the chain where the token is deployed, a `PultOFT` on each other chain mints and burns its representation, and the locked PULT must cover the supply of every `PultOFT` and the transfers in flight. On top of the standard LayerZero `OFT` and `OFTAdapter`, `PultFee` adds a default and per-destination fee capped at 5%, which the bridge contracts keep in a `feeBalance` for the owner together with the remainder below the 1e12-wei transfer unit. The bridge contracts also reject a zero recipient, a recipient equal to the destination bridge contract, and a send that would deliver nothing, and the PULT token lets its owner rescue tokens sent to the token contract.

Two auditors reviewed the code over one working day, on 28 October 2026. The review combined manual line-by-line analysis, proprietary AI-assisted analysis with manual triage of its candidates, a comparison of the fee logic and overrides against LayerZero's reference implementations, stateful fuzzing of a three-chain deployment with transfers in flight and delivered out of order, and a review of the staging deployment. The production bridge was not yet deployed at the time of the review.

Review Coverage

In addition to the audit team's general smart-contract security checklist, the review covered the following questions specific to the audited code:

- Does the PULT locked in the adapter, net of its `feeBalance`, cover the supply of every `PultOFT` and the transfers in flight once the fee and the remainder are kept at the source, including after direct transfers to the bridge contracts?
- Are the default and per-destination fees bounded by the 5% cap, and do `quoteOFT`, `quoteSend` and the `minAmountLD` check match what `send` debits and the destination credits?
- Can `withdrawFees` reach tokens other than the accumulated fees, in particular PULT that backs the `PultOFT` supply?
- Do the recipient and minimum-amount checks reject the cases they target in every direction, at quote and at send time, without blocking composer recipients or transfers between `PultOFT` contracts?
- Does PULT move one to one without callbacks, as the adapter's accounting assumes, and can the token owner's rescue function reach PULT held by the adapter?
- What can the owner change through the fee settings and the fee balance, and how does a change affect transfers already in flight?

Additional Attack Hypotheses

In addition to the issue described in this report, the audit examined the following hypotheses specific to the audited code.

- Whether `withdrawFees` on the adapter could pay out PULT that backs the `PultOFT` supply. `_debit` adds to `feeBalance` only the difference between the amount taken from the sender and the amount bridged, so withdrawing it leaves the backing intact.
- Whether a fee change while a transfer is in flight could change the delivered amount. The fee is charged only at the source, and the destination credits the amount encoded in the message.
- Whether fee rounding or the remainder could let a sender receive more than was debited or avoid the fee. The fee rounds down by less than one wei, and the bridged amount is the remainder after the fee rounded down to the transfer unit.
- Whether a stored per-destination fee could exceed the cap. `setFeeBps` checks the cap also for a disabled entry, so enabling it later cannot raise the fee above 5%.
- Whether a send could charge the sender and deliver nothing. `_debitView` rejects any send whose amount after the fee and the remainder is zero.
- Whether the PULT token could move the adapter's balance outside the bridge. PULT has no transfer hooks and no mint function, and `claimERC20` moves only tokens held by the token contract itself.

Scope Limitations

The audit was limited to the files listed in the scope. Tests, deployment scripts and configuration, the LayerZero and OpenZeppelin libraries, and the LayerZero endpoint, message libraries, DVNs and executors were not audited; they were reviewed only to establish whether the in-scope code works correctly in its interactions with them.

Deployment Requirements

- On every chain, `owner()` of each bridge contract and its delegate on the LayerZero endpoint must be the same intended multisig deployed on that chain, because the deployment scripts leave both roles with the deployer account.
- Exactly one `PultOFTAdapter` over PULT may be connected to the production bridge: every production `PultOFT` must list only the production adapter, whose `token()` is PULT, as its peer for the token's chain, and no other adapter over PULT, including the staging ones, may have a production bridge contract as its peer.
- The staging bridge, whose adapter also locks PULT and which is wired in every direction, should be retired: every staging `PultOFT` should have no supply left, and no staging bridge contract should have a peer set.
- For every direction, the number of block confirmations should exceed the depth to which the source chain can still reorganize, not only its sequencer's soft confirmation, because a reorganization after verification lets the destination credit a transfer whose source debit no longer exists; the reviewed configuration verifies transfers from one source chain after 5 blocks, about half a second.

Additional Notes

- The owner of each bridge contract sets its peers and fees, and its delegate on the LayerZero endpoint selects the libraries and DVNs that verify incoming transfers and can clear a verified transfer before delivery. `transferOwnership` and `renounceOwnership` do not change the delegate, so ownership must not be renounced: a renounced bridge contract keeps its delegate forever and leaves the accrued `feeBalance` unwithdrawable.
- The adapter's PULT balance can exceed the combined `PultOFT` supply plus its `feeBalance`, because anyone can transfer PULT to the adapter directly and the excess cannot be withdrawn. Backing checks and monitoring should therefore test `>=`: the equality that `scripts/e2e.ts` requires fails permanently after a 1-wei transfer.

Overall, the bridge is a small extension of LayerZero's `OFT` and `OFTAdapter` with a capped fee and input checks, and its lock-and-unlock accounting holds with the fee kept at the source; no path was found for an unprivileged account to take bridged PULT or mint unbacked PULT.

1.3 Project Overview

Summary

CLIENT	Catapult
CATEGORY	Bridge
PROJECT	PULT OFT Bridge
TYPE	Solidity
PLATFORM	EVM
TIMELINE	28.10.2026 - 02.11.2026

Scope of Audit

`contracts/CatapultTrade.sol` `contracts/PultFee.sol` `contracts/PultOFT.sol`
`contracts/PultOFTAdapter.sol`

Versions Log

DATE	COMMIT HASH	NOTE
28.10.2026	525c08247005b062a5e3ef054db893cbd0c9ca92	Initial Commit
02.11.2026	42cc9353841eea7cf5d11676bbffcb853ddc78b4	Re-audit Commit

Mainnet Deployments

CONTRACT	ADDRESS	BLOCKCHAIN
CatapultTrade	0x2eae2fef601c75d4ec24BdaDB17fE6e3D910c154	BSC
PultOFTAdapter	0xb4393f9404D2e49cB055C4E04AFE03c3a1fFf784	BSC
PultOFT	0xb4393f9404D2e49cB055C4E04AFE03c3a1fFf784	Arbitrum One
PultOFT	0xb4393f9404D2e49cB055C4E04AFE03c3a1fFf784	Robinhood Chain

The deployed bytecode of each contract matches the bytecode compiled from the deployment commit, whose contracts are identical to the re-audit commit, with the same compiler version and settings, including the metadata hash. The immutables are the LayerZero endpoint of each chain, the PULT token `0x2eae2fef601c75d4ec24BdaDB17fE6e3D910c154` as the adapter's token, and a decimal conversion rate of 1e12.

The deployment checks were verified on the contracts above as of 5 November 2026, 10:00 UTC (BSC block 131797058, Arbitrum One block 521443274, Robinhood Chain block 106652104):

- The `totalSupply` of PULT is 1,000,000,000 tokens, minted once at deployment, and the token contract has no function to mint more.
- The owner and the LayerZero delegate of all three bridge contracts are the deployer account, an externally owned account rather than a multisig. Both roles should be moved to a multisig on each chain, with `setDelegate` on the endpoint in addition to `transferOwnership`, which does not change the delegate.
- Each `PultOFT` has the adapter above, whose `token()` is PULT, as its peer for BSC, and the bridge contracts are peers of each other in every direction. The deployer account created no other contracts.
- The staging bridges are retired: no staging bridge contract has a peer set, and no staging `PultOFT` has any supply.
- The block confirmations exceed the source chain's reorganization depth only for transfers from BSC. Blocks of Arbitrum One and Robinhood Chain become final only once their batch is finalized on Ethereum, which took up to 4,492 and 12,041 blocks during a 40-minute measurement, while transfers from them are verified after 4,500 and 12,000 blocks. The confirmations from these two chains should be raised with a margin, for example to about 5,500 and 15,000 blocks.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

[1 INTRODUCTION]

1.6 Summary of Findings

Findings Count

0	0	0	1	1
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
L-1	Recipient check misses non-canonical to encodings	LOW	FIXED

[2 FINDINGS REPORT]

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 Recipient check misses non-canonical to encodings

LOW

FIXED

DESCRIPTION

PultOFT.sol:98-100 and PultOFTAdapter.sol:104-106 `_checkRecipient` compares the whole `bytes32` `to` with zero and with the destination peer, but the destination credits only its low 160 bits (OFTMsgCodec.sol:80-82). A `to` with non-zero upper bytes and zero low bits therefore passes the check and is minted to `0xdead` on a `PultOFT` chain (OFT.sol:83), or is never delivered on the adapter's chain, where `safeTransfer` to the zero address reverts (OFTAdapter.sol:96-105). Likewise, the destination bridge contract in the low bits receives the tokens itself, and a right-padding encoder such as ethers v6 `zeroPadBytes` sends them to an unowned address. In each case the source debit is final, and no role can return the tokens to the sender.

RECOMMENDATION

> We recommend making `_checkRecipient` reject any `to` with non-zero upper 96 bits and compare the decoded address with zero and with the decoded destination peer.

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io